

BigQuery: Google's Enterprise Data Warehouse

Core Concepts & Architecture

BigQuery is Google Cloud's fully managed, serverless data warehouse designed for analytics workloads. Unlike traditional databases focused on transactional operations, BigQuery specializes in analytical processing with these key characteristics:

- **Columnar Storage Architecture:** Data is stored by column rather than by row, making analytical queries dramatically faster as only relevant columns need to be read
- **Separation of Storage and Compute:** Resources scale independently based on workload demands
- **Serverless Design:** Zero infrastructure management required, automatic scaling
- **Massive Scalability:** Can query terabytes in seconds, petabytes in minutes
- **SQL Interface:** Supports standard SQL dialect (replacing older legacy SQL)

BigQuery Architecture Visualization

```

├─ Organization
│   └─ Project (container for billing, permissions)
│       └─ Dataset (container for tables, region-specific)
│           └─ Tables (actual data storage units)
│               └─ Columns (with schemas, data types)
│                   └─ Partitions (data segments)
│                       └─ Clusters (sorted data within partitions)
│                           └─ Views (logical tables defined by queries)
│                               └─ Functions (UDFs, stored procedures)
│                                   └─ Jobs (asynchronous operations)
│                                       └─ Reservations (slot allocations for capacity pricing)

```

How BigQuery Works Behind the Scenes

When you run a query in BigQuery:

1. The query is parsed and validated
2. A distributed execution plan is created
3. Worker nodes are dynamically assigned (slots)
4. Data is read from columnar storage, with irrelevant data skipped
5. Results are processed in memory across thousands of nodes
6. Results are returned or stored

Think of BigQuery as a massive parallel processor – unlike traditional databases that might use one or a few CPUs, BigQuery can leverage thousands of CPUs simultaneously across Google's data centers.

Data Storage & Table Types

Standard Tables

Standard tables are the most common type, with data stored directly in BigQuery's native columnar storage format. These offer:

- Fastest query performance
- Fully managed storage

- Integration with all BigQuery features
- Automatic data replication and 7-day change history

External Tables

External tables let you query data stored outside BigQuery without loading it first:

- **BigLake Tables:** Secure access to data in Cloud Storage, Amazon S3, or Azure Blob with table-level security
- **Object Tables:** For unstructured data like images, PDFs, etc.
- **Non-BigLake External Tables:** For structured data in Cloud Storage, Google Drive, Bigtable

External tables are excellent for:

- Frequently changing data that would require constant reloading
- Data you want to query occasionally without paying for BigQuery storage
- Maintaining a single source of truth with other systems

However, they typically have slower performance than native tables.

Views

Views are virtual tables defined by SQL queries:

- **Standard Views:** Run their defining query each time they're accessed
- **Materialized Views:** Cache results of their defining query in BigQuery storage for faster access, with automatic background refreshes

Views are particularly valuable for:

- Implementing column-level security
- Simplifying complex queries for end users
- Creating consistent business definitions across an organization

Table Feature Comparison

Feature	Standard Tables	External Tables	Views	Materialized Views
Storage Location	BigQuery	External Source	No Storage (Query)	BigQuery (Cached)
Query Speed	Fastest	Slowest	Recalculated Each Time	Fast (When Fresh)
Storage Costs	Yes	No	No	Yes
Update Frequency	Batch/Stream	Real-time	Real-time	Periodic Refresh
Best Use Case	Core Analytics	Data Lake Integration	Security, Simplification	Frequent Complex Queries

Data Optimization Techniques

Partitioning

Partitioning divides tables into smaller segments based on specific columns, dramatically improving query performance and reducing costs by scanning only relevant data portions.

Partition Types

1. Time-Unit Partitioning:

- **Daily** (default): Table divided by day, ideal for daily reporting

- **Hourly:** More granular for high-frequency data
- **Monthly/Yearly:** For long-term historical analysis

2. Ingestion-Time Partitioning:

- Based on when data was loaded into BigQuery
- Uses pseudo-column `_PARTITIONTIME`
- Ideal when your data lacks an explicit date column

3. Integer-Range Partitioning:

- Divides by integer column ranges
- **Example:** `PARTITION BY RANGE(customer_id BETWEEN 1 AND 1000, 1001 AND 2000, ...)`
- Good for numeric identifiers

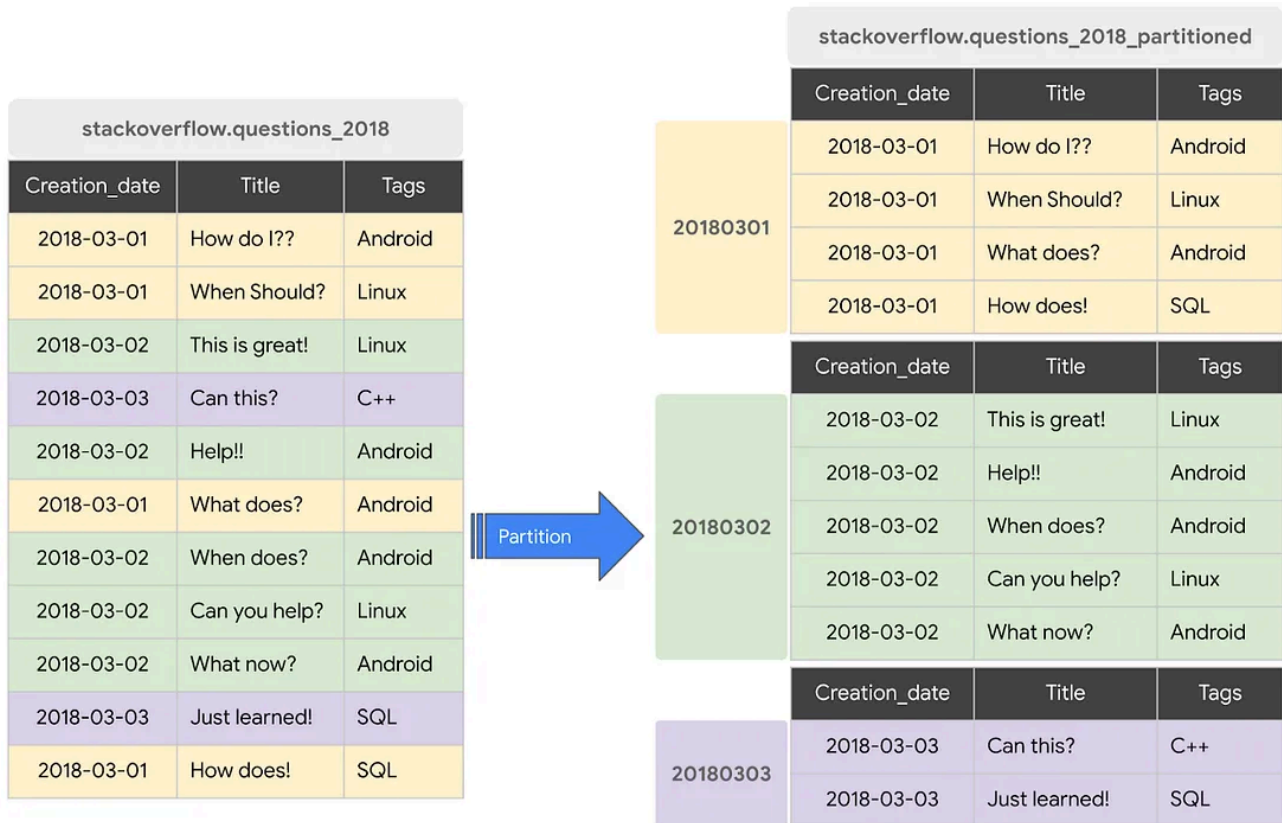
4. Column-Based Partitioning:

- Partitions based on any DATE or TIMESTAMP column
- **Example:** `PARTITION BY event_date`

Partitioning Example

```
-- Creating a time-partitioned table
CREATE TABLE dataset.sales_data
(
  transaction_id STRING,
  product_id STRING,
  sale_date DATE,
  amount NUMERIC
)
PARTITION BY sale_date
OPTIONS(
  partition_expiration_days=60, -- Auto-expire old partitions
  require_partition_filter=true -- Force partition pruning
);

-- Query leveraging partition pruning
SELECT SUM(amount) as daily_revenue
FROM dataset.sales_data
WHERE sale_date BETWEEN '2023-01-01' AND '2023-01-31'; -- Only scans January
partitions
```



Clustering

Clustering sorts data within partitions based on specified columns, further optimizing queries that filter or aggregate on those columns.

- Up to 4 clustering columns can be specified
- Columns order matters! Most selective should come first
- Automatically maintains sorting as data is added
- Most beneficial for high-cardinality columns (many distinct values)

```
-- Creating a partitioned and clustered table
CREATE TABLE dataset.user_events
(
  user_id STRING,
  event_type STRING,
  country STRING,
  event_date DATE,
  revenue NUMERIC
)
PARTITION BY event_date
CLUSTER BY country, event_type, user_id;

-- Query benefiting from both partition pruning and clustering
SELECT
  country,
  SUM(revenue) as total_revenue
FROM dataset.user_events
WHERE
  event_date BETWEEN '2023-01-01' AND '2023-01-31'
  AND country = 'US' -- Clustering allows fast filtering
GROUP BY country;
```

Partitioning vs. Clustering vs. Sharding

Feature	Partitioning	Clustering	Sharding (Legacy)
Purpose	Divide table into segments	Sort data within segments	Manually split tables
Implementation	Native BigQuery feature	Native BigQuery feature	Manual naming conventions
Maintenance	Automatic	Automatic	Manual
Maximum Segments	4,000 partitions	No limit on clusters	Manual limits
Best For	Date/time filtering	High-cardinality filtering	Legacy systems
Query Syntax	Uses normal WHERE clauses	Uses normal WHERE clauses	Requires wildcard syntax

When to Choose Which Approach:

- **Partitioning First:** Apply partitioning as your primary optimization when you frequently filter on a date/time column
- **Add Clustering:** Add clustering when you also frequently filter or aggregate on high-cardinality columns
- **Avoid Sharding:** Table sharding (e.g., tablename_20230101, tablename_20230102) is a legacy approach and generally inferior to partitioning

Clustered Tables

Orders table
Not clustered

Order_Date	Country	Status
2022-08-02	US	Shipped
2022-08-04	JP	Shipped
2022-08-05	UK	Canceled
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-05	US	Processing
2022-08-04	JP	Processing
2022-08-04	KE	Shipped
2022-08-06	UK	Canceled
2022-08-02	UK	Processing
2022-08-05	JP	Canceled
2022-08-06	UK	Processing
2022-08-05	US	Shipped
2022-08-06	JP	Processing
2022-08-02	KE	Shipped
2022-08-04	US	Shipped

Orders table
Clustered by Country

Order_Date	Country	Status
2022-08-02	US	Shipped
2022-08-05	US	Shipped
2022-08-05	US	Processing
2022-08-04	US	Shipped
2022-08-04	JP	Shipped
2022-08-04	JP	Processing
2022-08-05	JP	Canceled
2022-08-06	JP	Processing
2022-08-05	UK	Canceled
2022-08-06	UK	Canceled
2022-08-06	UK	Processing
2022-08-02	UK	Processing
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-04	KE	Shipped
2022-08-02	KE	Shipped

Orders table
Clustered by Country and Status

Order_Date	Country	Status
2022-08-05	US	Processing
2022-08-02	US	Shipped
2022-08-05	US	Shipped
2022-08-04	US	Shipped
2022-08-05	JP	Canceled
2022-08-04	JP	Processing
2022-08-06	JP	Processing
2022-08-04	JP	Shipped
2022-08-05	UK	Canceled
2022-08-06	UK	Canceled
2022-08-06	UK	Processing
2022-08-02	UK	Processing
2022-08-02	KE	Canceled
2022-08-06	KE	Shipped
2022-08-04	KE	Shipped
2022-08-02	KE	Shipped

Clustered and Partitioned Tables

Orders table
Not Clustered; Not partitioned

Order_Date	Country	Status
2022-08-02	US	Shipped
2022-08-04	JP	Shipped
2022-08-05	UK	Canceled
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-05	US	Processing
2022-08-04	JP	Processing
2022-08-04	KE	Shipped
2022-08-06	UK	Canceled
2022-08-02	UK	Processing
2022-08-05	JP	Canceled
2022-08-06	UK	Processing
2022-08-05	US	Shipped
2022-08-06	JP	Processing
2022-08-02	KE	Shipped
2022-08-04	US	Shipped

Orders table
Clustered by Country; Not partitioned

Order_Date	Country	Status
2022-08-04	JP	Shipped
2022-08-04	JP	Processing
2022-08-05	JP	Canceled
2022-08-06	JP	Processing
2022-08-06	KE	Shipped
2022-08-02	KE	Canceled
2022-08-04	KE	Shipped
2022-08-02	KE	Shipped
2022-08-05	UK	Processing
2022-08-06	UK	Canceled
2022-08-02	UK	Canceled
2022-08-06	UK	Processing
2022-08-02	US	Shipped
2022-08-05	US	Processing
2022-08-05	US	Shipped
2022-08-04	US	Shipped

Orders table
Clustered by Country; Partitioned by Order_Date (Daily)

	Order_Date	Country	Status
Partition: 2022-08-02	2022-08-02	KE	Shipped
	2022-08-02	KE	Canceled
Clusters: Country	2022-08-02	UK	Processing
	2022-08-02	US	Shipped

	Order_Date	Country	Status
Partition: 2022-08-04	2022-08-04	JP	Shipped
	2022-08-04	JP	Processing
Cluster: Country	2022-08-04	KE	Shipped
	2022-08-04	US	Shipped

	Order_Date	Country	Status
Partition: 2022-08-05	2022-08-05	JP	Canceled
	2022-08-05	UK	Canceled
Cluster: Country	2022-08-05	US	Shipped
	2022-08-05	US	Processing

	Order_Date	Country	Status
Partition: 2022-08-06	2022-08-06	JP	Processing
	2022-08-06	KE	Shipped
Cluster: Country	2022-08-06	UK	Canceled
	2022-08-06	UK	Processing

Data Loading & Export Options

Import Methods

1. Batch Loading (Free):

- **Web Console:** Up to 10MB from local files
- **Cloud Storage:** Preferred for large datasets
- **Command Line:** Using `bq load` utility
- **Supported Formats:** CSV (default), JSON, Avro, Parquet, ORC
- **Schema:** Can be auto-detected or explicitly defined

2. Streaming Inserts (Paid):

- **API-based:** Real-time data loading
- **Pub/Sub:** Via Dataflow templates
- **Dataflow:** For complex ETL
- **Trade-offs:** Higher cost, but immediate availability

3. External Data Sources (Federation):

- Cloud Storage, Google Sheets, Bigtable
- Query in place without loading
- Ideal for frequently changing data

Loading Example with Command Line

```
# Loading CSV from Cloud Storage with schema auto-detection
bq load \
  --autodetect \
  --source_format=CSV \
  mydataset.mytable \
  gs://mybucket/data.csv

# Loading Avro file (schema embedded in file)
bq load \
```

```
--source_format=AVRO \
mydataset.mytable \
gs://mybucket/data.avro
```

Export Options

1. Export Limitations:

- Maximum 1GB per file export
- Use wildcards for larger exports
- Formats: CSV, JSON, Avro only

2. Export Example:

```
# Export to multiple files in Cloud Storage
bq extract \
  mydataset.mytable \
  gs://mybucket/export_*.csv

# Export query results directly
bq query \
  --destination_table=mydataset.temp_results \
  --use_legacy_sql=false \
  'SELECT * FROM mydataset.mytable WHERE date = "2023-01-01"'

bq extract \
  mydataset.temp_results \
  gs://mybucket/query_results.csv
```

BigQuery Data Transfer Service

The BigQuery Data Transfer Service automates data loading from various sources:

- **Cloud Sources:** S3, Azure Blob Storage, Cloud Storage
- **Analytics Products:** Google Ads, Campaign Manager, Search Ads 360
- **SaaS Applications:** Salesforce, YouTube Channel
- **Database Migration:** Teradata, Redshift
- **Schedule Options:** One-time or recurring transfers

Important note: This service only transfers data *into* BigQuery, not out of it.

Cost Management & Optimization

Pricing Models

1. On-Demand Pricing:

- Pay per TB processed (first 1TB/month free)
- No minimum commitments
- Charges based on actual bytes read from columns, not result size
- Default and simplest option

2. Capacity (Slot-Based) Pricing:

- Purchase "slots" (virtual CPUs)
- Commitment options:
 - **Flex:** 60-second minimum commitment

- **Monthly:** 30-day commitment
- **Annual:** 365-day commitment (best rates)
- Ideal for predictable, high-volume workloads

Cost Control Strategies

1. Query Optimization:

- Select only needed columns (avoid `SELECT *`)
- Use partitioning filters
- Set a maximum bytes limit:

```
#standardSQL
-- Set maximum bytes processed
@{
  max_bytes_processed=1073741824
}
SELECT * FROM `mydataset.mytable` LIMIT 10;
```

2. Dry Run for Cost Estimation:

```
# Estimate cost before running query
bq query --dry_run --use_legacy_sql=false \
'SELECT * FROM `mydataset.huge_table`'
```

3. Project-Level Quotas:

- Set daily query bytes limit
- Avoid unexpected charges
- Set up budget alerts

4. Storage Optimization:

- Set table expiration
- Use time-travel (7-day history) instead of manual backups
- Compress data when possible

Performance Optimization

Query Patterns to Avoid

1. Self-Joins: Often can be replaced with window functions

```
-- Instead of self-join:
SELECT
  a.user_id,
  a.event_time as current_event,
  b.event_time as previous_event
FROM events a
JOIN events b
  ON a.user_id = b.user_id
  AND b.event_time < a.event_time

-- Better with window function:
```



```
SELECT
  user_id,
  event_time as current_event,
  LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) as
previous_event
FROM events
```

2. Cross-Joins: Generate massive result sets

```
-- Dangerous cross-join (cartesian product):
SELECT * FROM large_table_a, large_table_b

-- Better pre-aggregated approach:
SELECT
  a.category,
  COUNT(DISTINCT a.user_id) as users,
  AVG(b.metric) as avg_metric
FROM table_a a
JOIN table_b b ON a.user_id = b.user_id
GROUP BY a.category
```

3. Row-by-Row Updates: Avoid single-row operations

```
-- Instead of many single updates:
UPDATE mytable SET status = 'processed' WHERE id = 123;
UPDATE mytable SET status = 'processed' WHERE id = 124;

-- Better batch update:
UPDATE mytable
SET status = 'processed'
WHERE id IN (123, 124, 125, ... );
```

Advanced Optimization Techniques

1. Denormalize Data with Nested & Repeated Fields:

```
-- Create a table with nested and repeated fields
CREATE TABLE orders (
  order_id STRING,
  customer_id STRING,
  order_date DATE,
  items ARRAY<STRUCT<
    item_id STRING,
    quantity INT64,
    price NUMERIC
  >>
);

-- Query using nested fields
SELECT
  order_id,
  (SELECT SUM(item.price * item.quantity) FROM UNNEST(items) as item) as
```

```
order_total
FROM orders;
```

2. Use Approximate Functions for Large-Scale Aggregations:

```
-- Instead of exact count distinct (expensive):
SELECT COUNT(DISTINCT user_id) FROM events;

-- Use approximate count (much faster):
SELECT APPROX_COUNT_DISTINCT(user_id) FROM events;
```

3. Materialized Views for Frequent Complex Queries:

```
-- Create materialized view for common analysis
CREATE MATERIALIZED VIEW dataset.daily_revenue_by_product AS
SELECT
  DATE(transaction_time) as day,
  product_id,
  SUM(revenue) as daily_revenue
FROM dataset.transactions
GROUP BY 1, 2;
```

4. Query Structure Optimization:

- Join largest tables first
- Filter early to reduce data size
- Push complex operations to the end
- Only ORDER BY in the final output

Security & Access Control

Hierarchical Security Model

Security can be applied at multiple levels:

- **Organization:** Highest level for enterprises
- **Project:** Resource container with permissions
- **Dataset:** Collection of tables
- **Table/View:** Individual resource
- **Column:** Using policy tags for column-level security

Column-Level Security with Policy Tags

```
-- Create a policy tag taxonomy
-- (Usually done via UI or bq command)

-- Apply policy tags to columns during table creation
CREATE TABLE secure_data.employee_info
(
  employee_id STRING,
  name STRING,
  salary NUMERIC OPTIONS(policy_tags=['sensitive_financial']),
```

```
ssn STRING OPTIONS(policy_tags=['highly_restricted'])
);
```

Authorized Views

Authorized views allow sharing query results without giving access to source data.

```
-- Create view in source dataset
CREATE VIEW source_dataset.safe_view AS
SELECT
  non_sensitive_column1,
  non_sensitive_column2
FROM source_dataset.sensitive_table;

-- Grant view access to user/group without granting access to underlying table
-- (Usually done via UI or bq command)
```

Advanced Features

BigQuery BI Engine

- In-memory analysis service for BigQuery
- Sub-second query response for dashboards
- Integrates with Looker, Data Studio, Tableau
- Reserved memory capacity model

BigQuery ML

Transform BigQuery from just a data warehouse to a complete machine learning platform:

```
-- Create a linear regression model directly in SQL
CREATE MODEL `mydataset.price_prediction_model`
OPTIONS(
  model_type='LINEAR_REG',
  input_label_cols=['price']
) AS
SELECT
  square_footage,
  num_bedrooms,
  num_bathrooms,
  neighborhood,
  price
FROM `mydataset.housing_data`;

-- Use model for predictions
SELECT
  property_id,
  address,
  ML.PREDICT(MODEL `mydataset.price_prediction_model`,
    STRUCT(
      square_footage,
      num_bedrooms,
      num_bathrooms,
      neighborhood
    )
  )
```

```

).predicted_price AS estimated_price
FROM `mydataset.new_listings`;

```

BigQuery GIS

Geographic Information Systems capabilities:

```

-- Create points from latitude and longitude
SELECT
  store_id,
  ST_GEOGPOINT(longitude, latitude) AS location
FROM `mydataset.stores`;

-- Find stores within 10km of a location
SELECT
  store_id,
  ST_DISTANCE(
    ST_GEOGPOINT(longitude, latitude),
    ST_GEOGPOINT(-122.4194, 37.7749) -- San Francisco coordinates
  ) AS distance_meters
FROM `mydataset.stores`
WHERE ST_DWITHIN(
  ST_GEOGPOINT(longitude, latitude),
  ST_GEOGPOINT(-122.4194, 37.7749),
  10000 -- 10km in meters
);

```

Best Practices Summary

Architecture & Design

- Denormalize data where appropriate
- Use partitioning and clustering for large tables
- Consider materialized views for complex, frequent queries
- Balance streaming vs. batch loading based on needs

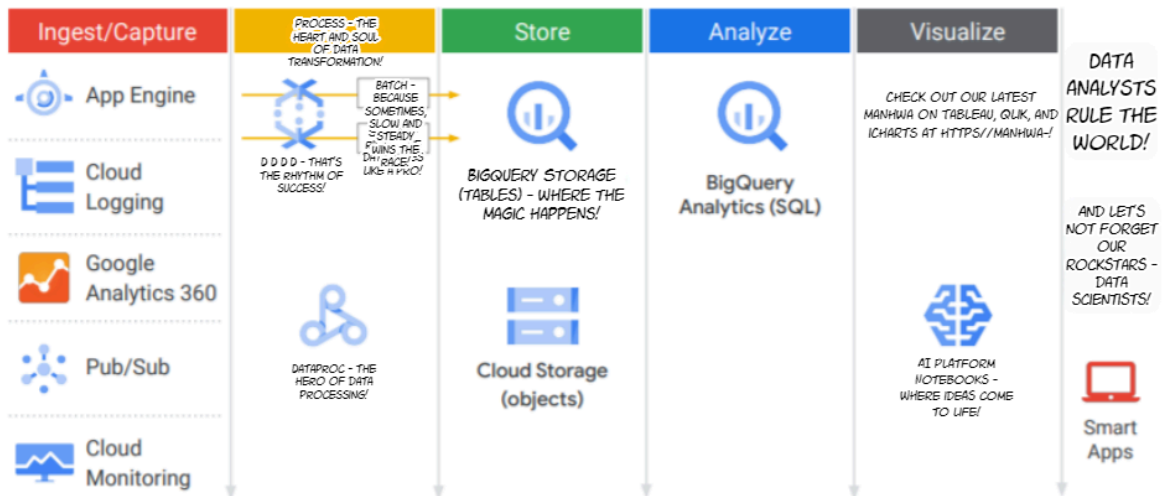
Query Efficiency

- Select only needed columns
- Filter on partitioned columns
- Use UNNEST for array operations
- Consider APPROX_ functions for large aggregations
- Avoid self-joins and cross-joins

Cost Management

- Use dry run to estimate costs
- Set project quotas and budget alerts
- Use table expiration for temporary data
- Consider capacity pricing for consistent workloads
- Cache commonly accessed data in BI Engine

Data processing solutions



Scaling streaming beyond BigQuery

- **Cloud Bigtable** : **Low latency/high-throughput**
 - 100,000 QPS at 6ms latency for a 10-node cluster
 - **More Cost-Efficient than** **Cloud Spanner** (~150 nodes)
- **BigQuery** : Easy, **inexpensive**
 - latency in order of seconds
 - 100k rows/second streaming

Resources for Further Learning

- [BigQuery Documentation](#)
- [Google Cloud Skills Boost - BigQuery Courses](#)
- [BigQuery ML Reference](#)
- [Query Performance Best Practices](#)